

Práctica 01

Nombre: Lara Guillén Gilabert

1. Alcanzabilidad

1.1. Forma canónica

El PDF con instrucciones ya presentaba el problema en forma canónica: describiendo el ejemplar y haciendo una pregunta sobre él.

Definición del problema

Dada una gráfica no dirigida $G = (V, E)$ con dos vértices distinguidos s y t . ¿Existe un camino de s a t en G que no repite vértices?

1.2. Diseño del algoritmo

El algoritmo no determinista a diseñar se divide en dos etapas principales: fase adivinadora y fase verificadora. La fase adivinadora toma un ejemplar E de un problema Π y adivina de forma no determinista una solución S . En la fase verificadora, se proporcionan E y S , y de manera determinista se determina si S es una solución válida para Π dado el ejemplar E .

1.2.1. Fase adivinadora

Podemos imaginar que tenemos un dado con el mismo número de caras que vértices en G . Hacemos un mapeo de la forma que a cada cara del dado le corresponda un vértice. Tiramos el dado al menos una vez, y decidimos que esa será la longitud de la secuencia que daremos como S . Luego, tiramos ese número de veces el dado y anotamos las caras que salen. De esa forma habremos construido una secuencia S , camino de G .

1.2.2. Fase verificadora

Es posible que S ni siquiera sea un camino válido para G , porque si G no es completa, podría estar usando aristas inexistentes, o que el inicio y/o final del S sean distintos a s y t . Primero, hay que verificar que el primer valor de la secuencia corresponde a s y el último a t . Si no corresponden, respondemos FALSO. Luego, hay que verificar que las aristas inducidas por la secuencia sean parte de G . Si al menos una de las aristas no es parte de G , respondemos FALSO. Por último, hay que verificar que ninguno de los vértices de la secuencia se repitan. Si hay al menos uno que se repite, respondemos FALSO. Si todas las verificaciones anteriores se superaron con éxito, entonces respondemos VERDADERO.

Notemos que el hecho de responder falso no significa que no exista ningún camino que cumpla con las características de la pregunta, simplemente decimos que el generado en la fase adivinadora no lo cumple.

También cabe mencionar que este adivinador es completamente ciego. Podríamos haber hecho uno que fije s como inicio y t como final, adivinando únicamente los vértices intermedios. De esa forma, la probabilidad de encontrar una secuencia que de verdadero como resultado a la pregunta aumentaría. Más aún, se podría forzar a que no se repitan aristas. Por ejemplo, si tuviéramos un saco con canicas que representan las aristas, podríamos ir sacando una por una sin volverlas a meter y así adivinamos un camino de forma no determinista pero haciendo que se cumpla la condición de no repetición. Aún así, no se podría asegurar que la secuencia dada sea válida para G . Habría que verificar que las aristas efectivamente existen.

Análisis de complejidad temporal y espacial:

1. Suponiendo que no se da, habría que contar el número de vértices de G . Podemos guardar en una variable el número.
 - Tiempo: $O(|V|)$
 - Espacio: $O(1)$
2. Tirar el dado para saber de qué tamaño construiremos la secuencia es constante. Para guardar el tamaño, podemos usar una variable.
 - Tiempo: $O(1)$
 - Espacio: $O(1)$
3. Tiramos el dado a lo más el número de vértices que tiene G , y vamos anotando la secuencia resultante, que igualmente es de tamaño a lo más el número de vértices de G .
 - Tiempo: $O(|V|)$
 - Espacio: $O(|V|)$

Esta fase toma tiempo $O(|V|)$ y espacio $O(|V|)$.

Análisis de complejidad temporal y espacial:

1. Determinar que el inicio y el final son s y t : son dos operaciones de tiempo constante. No se necesita espacio adicional.
 - Tiempo: $O(1)$
 - Espacio: $O(1)$
2. Verificar que todas las aristas de S son también aristas de G : el camino más grande que produce la fase adivinadora es del número de vértices de G . Hay que verificar $|V| - 1$ cantidad de aristas. No se necesita espacio adicional.
 - Tiempo: $O(|V|)$
 - Espacio: $O(1)$
3. Verificar que ningún vértice puede hacerse ordenando la secuencia de vértices S y haciendo un recorrido lineal que cheque que ningún índice es igual a su siguiente. Si no queremos destruir la secuencia original, habría que copiarla.
 - Tiempo: $O(|V| \log |V|)$
 - Espacio: $O(|V|)$

Esta fase toma tiempo $O(|V| \log |V|)$ y espacio $O(|V|)$.

Entonces, el algoritmo en total toma tiempo $O(|V| \log |V|)$ y espacio $O(|V|)$. Es un algoritmo de espacio y tiempo polinomial.

1.3. Implementación**1.3.1. Generación de ejemplares**

Implementa de forma modificada el algoritmo planteado previamente. Primero, necesitamos construir un ejemplar de gráfica, acotada entre 10 y 20 vértices. Para ello, se elige un tamaño al azar de entre 10 y 20. Se crea una matriz vacía de $n \times n$ y se recorre únicamente el triángulo superior, llenando de forma aleatoria algunas celdas con 1 (como si lanzáramos una moneda). De forma aleatoria se eligen también dos vértices s

y t de la matriz. De esta forma generamos un ejemplar del problema. La gráfica es representada mediante su matriz de adyacencia. Se asume que además de no dirigida, es simple y por eso la diagonal principal se llena con ceros. Debido a que se tiene una matrix $n \times n$, construir un ejemplar toma $O(n^2)$ en tiempo y espacio.

```

1 def generar_matriz_adyacencia():
2     n = random.randint(10, 20)
3     matriz = [[0] * n for _ in range(n)]
4
5     for i in range(n):
6         for j in range(i + 1, n):
7             valor = random.randint(0, 1)
8             matriz[i][j] = valor
9             matriz[j][i] = valor
10
11     vertice_s, vertice_t = random.sample(range(n), 2)
12
13     return matriz, (vertice_s, vertice_t)

```

1.3.2. Fase adivinadora

Se elige una longitud al azar, no mayor que n o menor que 1 y se eligen al azar los vértices de la secuencia. Es el análogo en Python de tirar los dados. Construir una solución toma $O(n)$ en tiempo y espacio.

```

1 def generar_camino(matriz):
2     n = len(matriz)
3     longitud = random.randint(1, n)
4     camino = [random.randrange(n) for _ in range(longitud)]
5
6     return camino

```

1.3.3. Fase verificadora

Realiza las verificaciones mencionadas en el diseño del algoritmo. Regresa True sólo si pasa las tres. Para las repeticiones, toma de hecho tiempo lineal, porque se verifica comparando $\text{len}(\text{set}(\text{camino}))$, $\text{len}(\text{camino})$.

```

1 def verificar_camino(matriz, camino, s, t):
2     if camino[0] != s or camino[-1] != t:
3         return False
4
5     for i in range(len(camino) - 1):
6         actual, siguiente = camino[i], camino[i + 1]
7         if matriz[actual][siguiente] == 0:
8             return False
9
10    if len(set(camino)) != len(camino):
11        return False
12
13    return True

```

El resto de funciones son de impresión.

1.3.4. Instrucciones y ejemplo de ejecución

El programa en Python `alcanzabilidad.py` se ejecuta de la siguiente forma:

```
1 python alcanzabilidad.py
```

No necesitamos pasarle parámetros por línea de comandos, así que simplemente ejecutando el programa obtenemos algo así:

```
1 Numero de vertices: 14
2
3      0  1  2  3  4  5  6  7  8  9 10 11 12 13
4 0:  0  0  1  1  0  1  1  1  0  0  0  1  1  1
5 1:  0  0  1  0  0  1  0  1  1  0  0  1  1  0
6 2:  1  1  0  1  0  0  1  1  1  0  0  0  1  0
7 3:  1  0  1  0  0  1  1  1  0  0  0  0  0  0
8 4:  0  0  0  0  0  1  0  0  0  1  1  1  1  1
9 5:  1  1  0  1  1  0  0  1  1  0  1  1  1  0
10 6:  1  0  1  1  0  0  0  1  1  0  0  1  1  0
11 7:  1  1  1  1  0  1  1  0  1  1  0  0  0  1
12 8:  0  1  1  0  0  1  1  1  0  0  1  0  1  0
13 9:  0  0  0  0  1  0  0  1  0  0  1  1  1  0
14 10: 0  0  0  0  1  1  0  0  1  1  0  1  0  0
15 11: 1  1  0  0  1  1  1  0  0  1  1  0  1  1
16 12: 1  1  1  0  1  1  1  0  1  1  0  1  0  0
17 13: 1  0  0  0  1  0  0  1  0  0  0  1  0  0
18
19 Vertices seleccionados: s=3, t=10
20 Camino aleatorio: [4]
21 Es un camino valido y sin repeticiones de s a t: False
```

2. 3-SAT

En este caso no se indicó de forma explícita la forma canónica del problema, así que lo formulé basándome en el libro de Garey [1].

2.1. Forma canónica

Definición del problema

Dada una colección $C = \{c_1, c_2, \dots, c_m\}$ de cláusulas sobre un conjunto finito U de variables, con exactamente tres variables cada una. ¿Existe una asignación de verdad para U que satisfaga todas las cláusulas en C ?

2.2. Diseño del algoritmo

De nuevo, tenemos que diseñar un algoritmo no determinista con una fase adivinadora y una verificadora.

2.2.1. Fase adivinadora

Podemos imaginar que tenemos una moneda. Hacemos un mapeo de la forma que si la moneda cae cara, se toma como verdadero, y si cae cruz, se toma como falso. Tiramos la moneda una vez por cada variable del ejemplar E y anotamos el resultado de cada tiro. De esa forma habremos construido una secuencia S , asignación de valores de verdad para las variables de E .

2.2.2. Fase verificadora

Es posible que S no satisfaga a E , porque puede haber alguna cláusula en la que los tres literales resulten falsos con los valores de S . Hay que verificar cada cláusula de E . Para cada literal de la cláusula, se toma el valor que S le asigna a su variable; si el literal está negado, se toma el valor contrario. Si al menos uno de los tres literales resulta verdadero, la cláusula está satisfecha. Si ninguno lo es, respondemos FALSO. Si todas las cláusulas están satisfechas, entonces respondemos VERDADERO.

Notemos que el hecho de responder falso no significa que no exista ninguna asignación que satisfaga a E , simplemente decimos que la generada en la fase adivinadora no lo cumple.

Análisis de complejidad temporal y espacial:

1. Suponiendo que no se da, habría que contar el número de variables de E . Podemos guardar en una variable el número.
 - Tiempo: $O(n)$
 - Espacio: $O(1)$
2. Tirar la moneda para saber el valor de una variable es constante. Para guardar el valor, podemos usar una variable.
 - Tiempo: $O(1)$
 - Espacio: $O(1)$
3. Tiramos la moneda tantas veces como variables tiene E , y vamos anotando la secuencia resultante, que es de tamaño igual al número de variables de E .
 - Tiempo: $O(n)$
 - Espacio: $O(n)$

Esta fase toma tiempo $O(n)$ y espacio $O(n)$.

Análisis de complejidad temporal y espacial:

1. Evaluar un literal de una cláusula: consultar el valor de su variable en S y negarlo si corresponde. Cada cláusula tiene tres literales, así que son a lo más tres operaciones de tiempo constante. No se necesita espacio adicional.
 - Tiempo: $O(1)$
 - Espacio: $O(1)$
2. Verificar que todas las cláusulas de E estén satisfechas: hay que evaluar las m cláusulas de E . No se necesita espacio adicional.
 - Tiempo: $O(m)$

- Espacio: $O(1)$

Esta fase toma tiempo $O(m)$ y espacio $O(1)$ adicional.

Entonces, el algoritmo en total toma tiempo $O(n + m)$ y espacio $O(n)$. Es un algoritmo de espacio y tiempo polinomial.

2.3. Implementación

2.3.1. Generación de ejemplares

Implementa de forma modificada el algoritmo planteado previamente. Primero, necesitamos construir un ejemplar de 3SAT, con un número de variables acotado entre 10 y 50, y un número de cláusulas acotado entre 5 y 25. La cota máxima la elegí yo en pro de que fuera legible tanto el ejemplar como la asignación. Para ello, se elige al azar el número de variables n y el número de cláusulas m . Se les asignan nombres x , y y z a las variables. Para cada cláusula, se eligen al azar tres variables distintas y, para cada una, de forma aleatoria (como si lanzáramos una moneda) se decide si aparece negada o no. De esta forma generamos un ejemplar del problema. Cada cláusula es representada mediante una lista de tres pares (variable, negada). Construir un ejemplar toma $O(n + m)$ en tiempo y espacio.

```

1 def generar_ejemplar_3sat():
2     n = random.randint(10, 50)
3     m = random.randint(5, 25)
4     nombres = generar_nombres(n)
5
6     clausulas = []
7     for _ in range(m):
8         variables = random.sample(range(n), 3)
9         clausula = [(v, random.choice([True, False])) for v in variables]
10        clausulas.append(clausula)
11
12    return nombres, clausulas

```

2.3.2. Fase adivinadora

Se tira una moneda por cada variable del ejemplar. Es el análogo en Python de tirar la moneda. Construir una solución toma $O(n)$ en tiempo y espacio.

```

1 def generar_asignacion(n):
2     return [random.choice([True, False]) for _ in range(n)]

```

2.3.3. Fase verificadora

Realiza las verificaciones mencionadas en el diseño del algoritmo. Regresa True sólo si todas las cláusulas son satisfechas. Para cada cláusula, se detiene en cuanto encuentra un literal verdadero, y si ninguno lo es, regresa False.

```

1 def verificar_asignacion(clausulas, asignacion):
2     for clausula in clausulas:
3         satisfecha = False
4         for v, neg in clausula:
5             valor = not asignacion[v] if neg else asignacion[v]

```

```

6         if valor:
7             satisfecha = True
8             break
9         if not satisfecha:
10            return False
11
12    return True

```

El resto de funciones son de impresión.

2.3.4. Instrucciones y ejemplo de ejecución

El programa en Python `tres_sat.py` se ejecuta de la siguiente forma:

```

1 python tres_sat.py

```

No necesitamos pasarle parámetros por línea de comandos, así que simplemente ejecutando el programa obtenemos algo así:

```

1
2 Numero de variables: 35
3 Numero de clausulas: 8
4
5 (z4 v ~y5 v ~y1)
6 (y9 v x4 v y8)
7 (~x3 v ~y12 v ~y8)
8 (y3 v y11 v z11)
9 (x6 v z9 v z5)
10 (z11 v ~z7 v ~y5)
11 (~y11 v x6 v ~y8)
12 (~x1 v x8 v y12)
13
14 Asignacion de variables:
15 x1 = False
16 y1 = True
17 z1 = False
18 x2 = False
19 y2 = True
20 z2 = False
21 x3 = True
22 y3 = False
23 z3 = True
24 x4 = True
25 y4 = False
26 z4 = True
27 x5 = False
28 y5 = False
29 z5 = True
30 x6 = True
31 y6 = False
32 z6 = True
33 x7 = False

```

```
34 y7 = False
35 z7 = True
36 x8 = False
37 y8 = False
38 z8 = True
39 x9 = True
40 y9 = True
41 z9 = False
42 x10 = True
43 y10 = True
44 z10 = False
45 x11 = False
46 y11 = False
47 z11 = True
48 x12 = False
49 y12 = False
50
51 Verdadero
```

Bibliografía

- [1] Michael R. Garey y David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. 1979. ISBN: 0-7167-1044-7.